

AI Research Pipeline

A LangChain Multi-Agent System — Architecture, Flow Charts & Technology Breakdown

This report walks through **pipeline.py** and **tools.py**: a four-stage automated research pipeline that takes a topic, searches the web, scrapes the best source, drafts a report, and critiques it.

This report is based on the full, confirmed contents of all three files: **pipeline.py**, **tools.py**, and **agents.py**. All descriptions below reflect the actual code — nothing is inferred.

1. What This Project Does

The project is an automated research assistant. Given a topic typed by the user, it runs a 4-stage pipeline that mimics how a human researcher would work: search the web, read the most promising source in depth, write a structured report, then have a second pass critique that report for gaps or weaknesses.

It is built with **LangChain** (agent framework + tool abstractions), uses the **Tavily** search API for live web search, and **BeautifulSoup** + **requests** for scraping full page content.

1.1 Overall Pipeline Flow

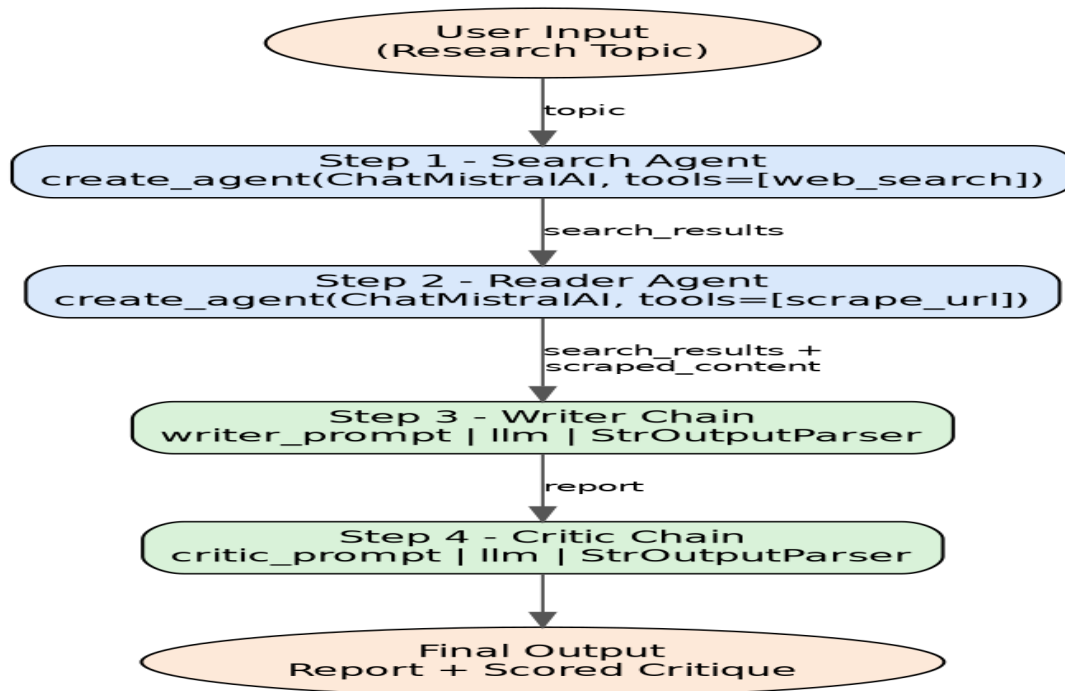


Figure 1 — End-to-end flow from user topic to final report + critique.

Each stage's output feeds directly into the next stage's input — a classic linear "chain of agents" pattern, orchestrated by the plain Python function `run_research_pipeline()` rather than a graph framework like LangGraph.

2. Stage-by-Stage Breakdown

Stage	Component	Type	Purpose
1	Search Agent	<code>create_agent(ChatMistralAI, tools=[WebSearch()])</code>	Finds relevant sources for the topic via Tavily
2	Reader Agent	<code>create_agent(ChatMistralAI, tools=[ScrapeURL()])</code>	Picks the best URL from search results and scrapes full text
3	Writer Chain	<code>writer_prompt llm StrOutputParser()</code>	Synthesizes research into a structured report
4	Critic Chain	<code>critic_prompt llm StrOutputParser()</code>	Scores the report out of 10 with structured feedback

2.1 Step 1 — Search Agent

`build_search_agent()` calls LangChain's `create_agent()` (the modern prebuilt agent constructor) bound to a single tool, `web_search`, and the shared `ChatMistralAI` LLM. It's invoked with a message asking it to find "recent, reliable and detailed information" on the topic. The agent's final message content is stored as `state['search_results']`.

2.2 Step 2 — Reader Agent

`build_reader_agent()` builds a second `create_agent()` instance with the `scrape_url` tool. It receives the first 800 characters of the search results and is instructed to pick the most relevant URL and scrape it. The scraped output becomes `state['scraped_content']`.

2.3 Step 3 — Writer Chain

Not an agent — a plain LCEL chain: `writer_prompt | llm | StrOutputParser()`. It receives the topic plus a combined string of the search results and scraped content, and is instructed to produce a report in a fixed structure: Introduction, Key Findings (minimum 3 points), Conclusion, and Sources (all URLs found). No tool calls happen here — it's pure generation grounded in the gathered research, stored as `state['report']`.

2.4 Step 4 — Critic Chain

Also a plain LCEL chain: `critic_prompt | llm | StrOutputParser()`. It takes the finished report and is instructed to respond in a fixed format: a numeric Score: X/10, a bulleted Strengths list, a bulleted Areas to Improve list, and a one-line verdict. Stored as `state['feedback']`. This acts as an automated, structured quality-control pass before the report reaches the user.

2.5 Shared LLM & Agent Wiring

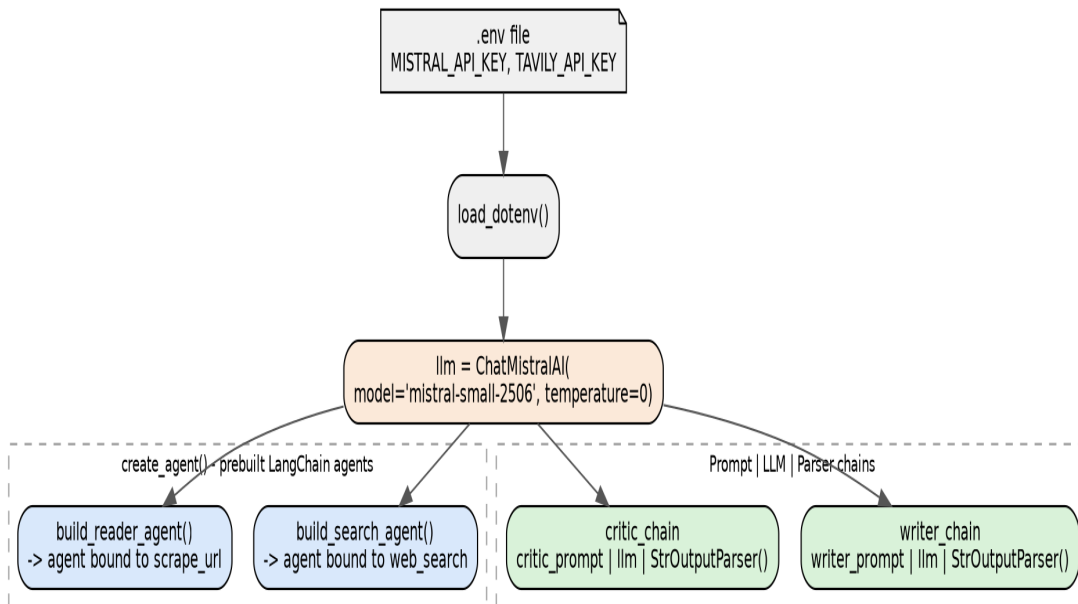


Figure 1b — All four components share one `ChatMistralAI` instance loaded from `.env`.

All four components — both agents and both chains — share a single `llm = ChatMistralAI(model='mistral-small-2506', temperature=0)` instance. Temperature 0 makes outputs deterministic/repeatable rather than creative, which suits research and critique tasks. Credentials (Mistral + Tavily API keys) are loaded once via `load_dotenv()`.

2.6 Prompt Output Structures

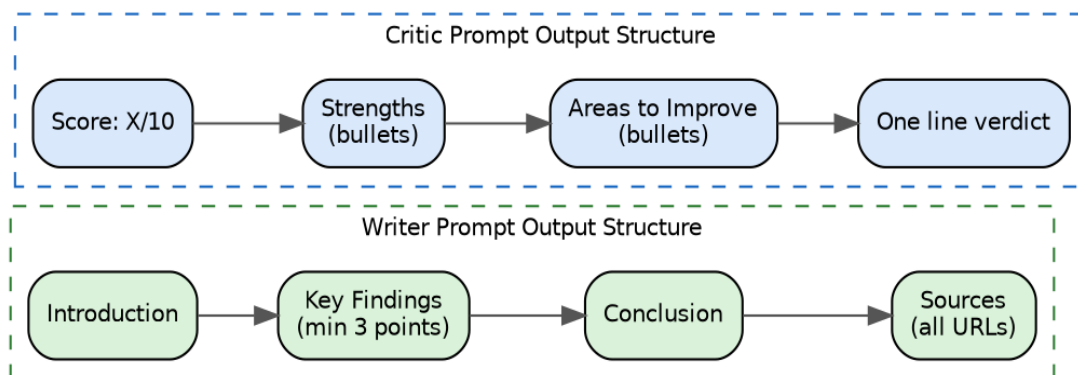


Figure 1c — The fixed output sections each prompt enforces via its template.

3. Tool-Level Flow Charts (tools.py)

Both agents (Search, Reader) are only as good as the tools they're given. `tools.py` defines exactly two `@tool`-decorated functions consumed by those agents.

3.1 web_search Tool

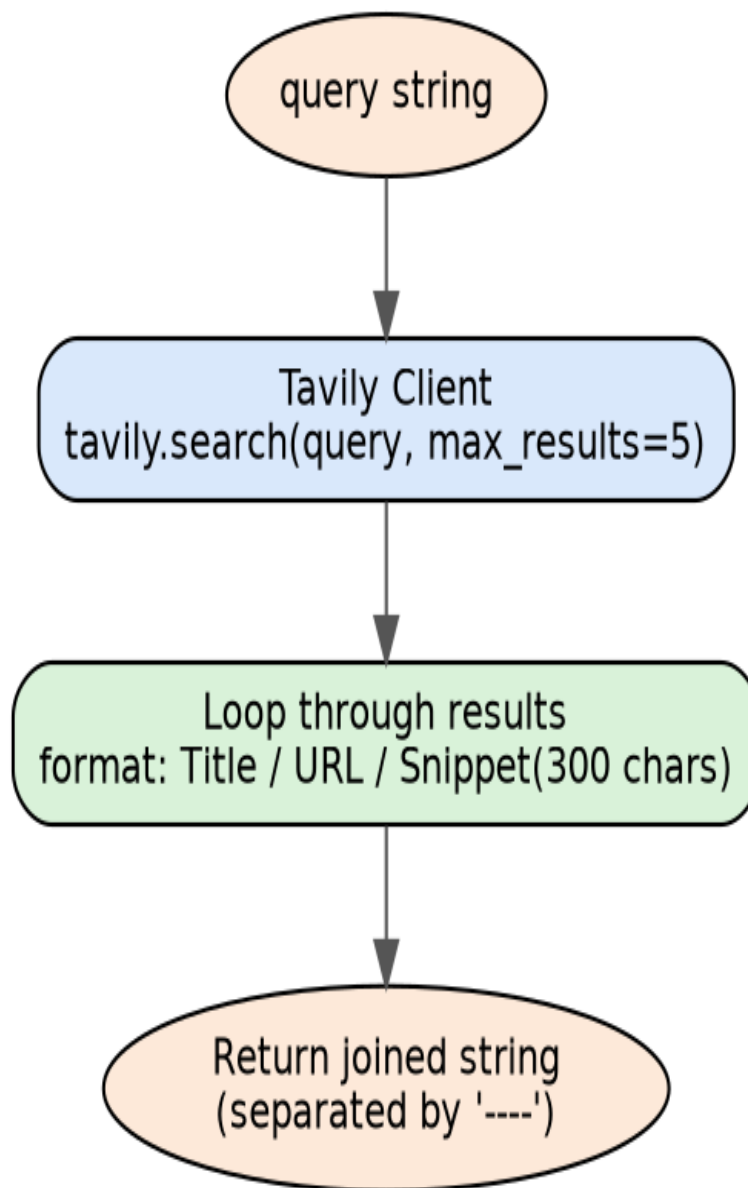


Figure 2 — `web_search`: query in, formatted Tavily results out.

Calls the Tavily API (`tavily.search()`) for up to 5 results, then formats each as Title / URL / a 300-character snippet, joined into one string. The Tavily API key is loaded from environment variables via

python-dotenv.

3.2 scrape_url Tool

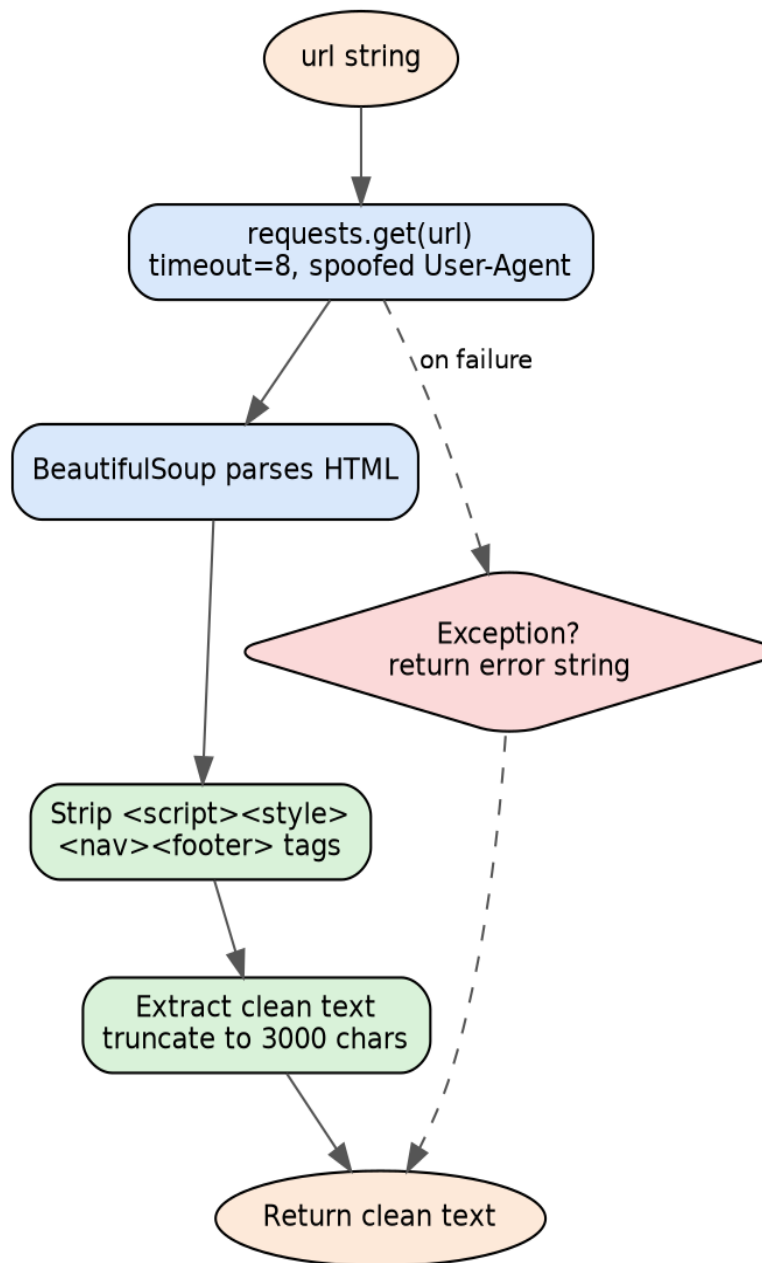


Figure 3 — `scrape_url`: URL in, cleaned page text out (or an error string).

Fetches the page with `requests` (8-second timeout, spoofed User-Agent to avoid basic bot-blocking), parses it with `BeautifulSoup`, strips non-content tags (`script/style/nav/footer`), and returns up to 3000 characters of clean text. Wrapped in a `try/except` so a failed scrape returns an error message instead of crashing the pipeline.

4. State / Data Flow (pipeline.py)

The whole pipeline is coordinated through a single Python dict, `state`, which accumulates one new key after each stage completes. This is the simplest possible orchestration pattern — no shared memory object, no graph state machine, just sequential dict writes.

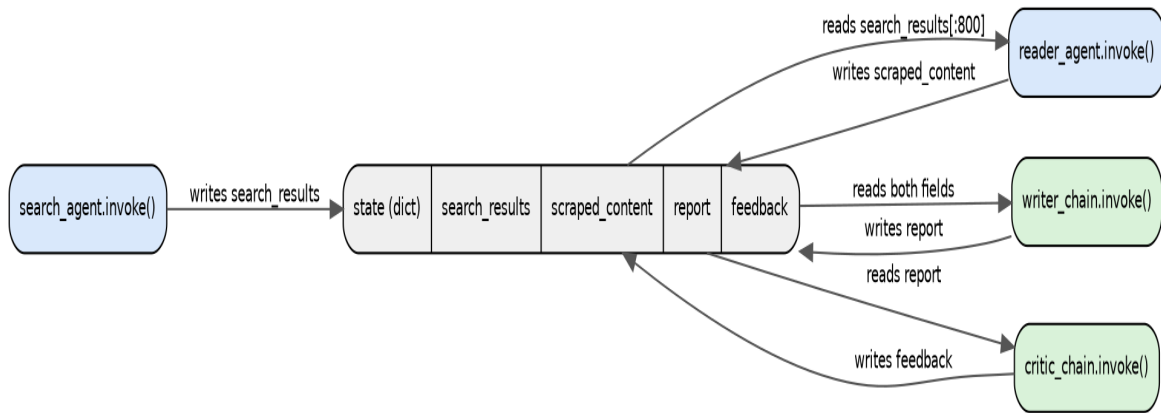


Figure 4 — How the shared state dict is built up across the four stages.

5. Technologies Used & Why

Technology	Role in the Project	Why It Was Chosen
LangChain (create_agent)	Builds the Search & Reader tool-using agents	Modern prebuilt agent constructor — handles the tool-call loop without
LangChain (LCEL)	writer_chain / critic_chain via prompt llm parser	Composable, minimal-boilerplate way to wire a prompt template straight
Mistral AI (ChatMistralAI)	The LLM powering all 4 stages, model mistral-small2.5-7b	Accessible, cost-efficient model; temperature=0 keeps agent/chain output
Tavily API	Web search backend for the search_agent	Purpose-built search API for LLM/RAG use cases — returns clean, structured
BeautifulSoup	HTML parsing in scrape_url	De-facto standard for extracting readable text from arbitrary web pages
requests	HTTP fetching in scrape_url	Simple, reliable synchronous HTTP client for one-off page fetches
python-dotenv	Loads MISTRAL / TAVILY API keys from .env	Keeps secrets out of source code, in both agents.py and tools.py

6. Design Observations

- Linear, not graph-based: the pipeline is a straight function call sequence, not LangGraph — easy to read, but no branching, retries, or conditional routing between stages.
- One shared LLM instance: all four stages reuse the same ChatMistralAI(temperature=0) object, so behavior differences between stages come entirely from the prompts/tools, not model config — a clean, consistent design choice.
- Truncation choices: search results are cut to 800 chars before being shown to the Reader Agent (keeps the prompt small), and scraped content is capped at 3000 chars (keeps the Writer's context bounded).
- Error handling exists in scrape_url (try/except) but not around the agent .invoke() calls in pipeline.py, nor around create_agent's tool-calling loop — a Mistral/Tavily API failure or rate limit would currently crash the whole run.
- Two .env dependencies: MISTRAL_API_KEY (used implicitly by ChatMistralAI) and TAVILY_API_KEY (used explicitly in tools.py) both need to be present, but only tools.py checks for the key via os.getenv — a missing MISTRAL_API_KEY will surface as an unclear error from the Mistral client rather than a clear message.
- The Writer and Critic prompts enforce strict output formats (fixed sections / Score: X/10), which makes their outputs easy to parse programmatically later, but nothing in pipeline.py currently validates that the LLM actually followed that format.